

2026年6月版

Loop Engineering

别再问我什么是循环工程

从一句句喂 prompt，到设计让 agent 自己干活的系统

The New Layer Above the Harness

文档版本：v260615

发布时间：2026-06-15 (build #0)

涵盖内容：定义 · 演化 · 解剖 · 六零件 · 评判器 · 真实案例 · 代价 · 上手

花叔

花叔

本手册包含的产品信息、功能描述和定价可能随时变化，请以各产品官方文档为准。

目录

CONTENTS

第一部分 · 它是什么

§01 别再问了：Loop Engineering 到底是什么

§02 三级跳：从 Prompt 到 Context 到 Loop

第二部分 · 它怎么转

§03 一个循环的五个动作

§04 六个零件：搭一个 Loop 需要什么

§05 生成器与评判器：为什么写代码的 AI 不能给自己打分

第三部分 · 它在哪跑、要什么代价

§06 让循环在你睡觉时跑：三个真实的 Loop

§07 代价：验证债、理解腐烂、token 失控

第四部分 · 你怎么开始

§08 当工程师，不只是按下启动键

§09 今天就动手：搭你的第一个 Loop

§01 别再问了：Loop Engineering 到底是什么

Stop Asking: What Loop Engineering Really Is

又一个 XX Engineering? 这次的不同在于，它不是教你怎么干活，而是让你别再亲自干活。

又来一个

Prompt Engineering 的课还在卖，Context Engineering 的墨迹没干，Harness Engineering 我刚写完一本书。现在 Loop Engineering 又来了。

过去一年「XX Engineering」冒出来一串，造词速度快赶上模型迭代。你想翻白眼，我理解。

但这一个有点不一样。它不是教你把活干得更好，它是让你彻底从那个干活的位置上退出来。

前面几个词都还假设你坐在键盘前，一句句指挥 agent。Loop engineering 把这个假设删掉了。你不再在循环里，你在循环外面，负责造那个循环。

一句话定义

命名这个词、把它写成文章的人，是 Addy Osmani，Google Chrome 团队的工程师。他给的定义很短，我直接放原话。

原文：Loop engineering is replacing yourself as the person who prompts the agent. You design the system that does it instead.

翻成中文：循环工程，就是把「那个负责 prompt agent 的人」从你自己换成一套系统。你不再亲自一句句喂，而是设计那套替你喂的系统。

这句话的重心，在「替换你自己」。不是把提示词写得更好，也不是把上下文管得更精，是把你这个人从那个位置上挪走。

这是一个位置的转移。以前你是发动机，现在你是设计发动机的人。你写的不再是给 agent 的话，是一个会自动给 agent 发话的东西。

一周之内，三个人点着了它

这个词不是某天某个人拍脑袋造出来的。它是 2026 年 6 月那一周，几拨人几乎同时撞到了同一件事，然后才有了名字。

把它真正引爆的，是 Peter Steinberger（OpenClaw 的作者）6 月发的一条推。这条推一下冲到 800 多万浏览。

原文： You shouldn't be prompting coding agents anymore. You should be designing loops that prompt your agents.

你不该再去 prompt 编程 agent 了。你该去设计那些会 prompt 你 agent 的循环。

几乎同时，**Boris Cherny**（Anthropic 的人，Claude Code 的负责人）也在说同一件事。

原文： I don't prompt Claude anymore. I have loops running that prompt Claude and figuring out what to do. My job is to write loops.

我不再 prompt Claude 了。我让一堆循环跑着，由它们去 prompt Claude、去琢磨该干什么。我的工作，就是写循环。

真正给它命名的，是 **Addy Osmani**，Google Chrome 团队的工程师。6 月 7 日他在个人博客把这件事写成文章，标题就叫 Loop Engineering，顺手把 Steinberger 和 Cherny 的话都引了进去，第二天同步到 Substack。一个引爆，一个同声，一个命名，前后就一周。

这三句话其实在说同一件事

把三个人的话叠在一起看，你会发现他们指的是同一个动作。

Cherny 说「我的工作就是写循环」。Steinberger 说「去设计那些循环」。Osmani 说「设计那套替你喂的系统」。关键词都是同一个：你设计的对象，从 agent 的一次行为，变成了驱动 agent 的整个系统。

这就是为什么三个互不通气的人，会在一周里指向同一个词。不是巧合，是大家各自做着类似的事，做到一个程度，发现需要一个共同的名字才能交流。

就像当年的 vibe coding。你可以笑它土，但它确实把一种做法变成了能讨论的东西。Loop engineering 是同一回事。

从「你 prompt agent」到「你设计 prompt agent 的系统」

我把这个转变拆成最简单的两句话，方便你记住。

旧世界：你坐在那儿，一句句 prompt agent。它干完一件，停下，等你下一句。你是这个循环里的人肉时钟，每一拍都得你来敲。

新世界：你设计一套东西，让它自己一拍一拍地敲。它在定时器上跑、自己孵化小帮手去干活、自己把结果喂回给自己。你不在循环里了，你在循环外面看着它转。

Addy 后面那句话把这层关系说得很准。

原文： Loop engineering sits one floor above the harness.

循环工程，坐在 harness 的上一层楼。下面那层 harness 负责武装 agent 的单次运行，上面这层 loop 负责让它一遍遍自己跑起来。这层楼怎么搭，下一章细说。

到这儿你大概能感觉到，这不是又一个换皮的新词。它标记的是一次身份变化：**从操作 agent 的人，变成调度 agent 的人。**

你以前的价值在于「会指挥」。现在指挥这件事被你自己写的循环接管了。你的价值，转移到了「会造循环、会给循环装一个能说不的检查」上面。后面这点为什么是整件事最难的地方，我们慢慢讲。

§02 三级跳：从 Prompt 到 Context 到 Loop

The Triple Jump: From Prompt to Context to Loop

四层栈，一层管一件事。看清边界，你就知道 loop 站在哪、为什么它是上一层楼。

四层栈，先摆出来

过去两年 AI 圈造的那串「XX Engineering」，其实不是互相替代的，是叠起来的。一层一层往上摞，每层管一件更大的事。

我先把这张表摆出来，后面逐层讲。

层	管什么	核心问题
Prompt engineering	写好一次的提示词	我该告诉模型什么
Context engineering	这一刻窗口里放什么	检索什么、摘要什么、清掉什么
Harness engineering	单次运行的武装	给哪些工具、允许哪些动作、什么算完成
Loop engineering	在 harness 之上调度	怎么让它自己一遍遍跑起来

每往上一层，你操心的东西就大一号。从「一句话」到「一个窗口」，再到「一次运行」，最后到「一整套自动跑的循环」。下面挨个说。

第一层：Prompt，你写给模型的那句话

最底层，也是大家最熟的。Prompt engineering 管的是「我该告诉模型什么」。

你斟酌措辞、给例子、定角色、调语气。它的边界很清楚：就是这一次，你递给模型的那段文字。递进去，模型吐出来，结束。

这层的麻烦在于，它假设你每次都在场，每次都亲自递。一个 prompt 解决一件事，下一件还得你再写一个。

第二层：Context，这一刻窗口里放什么

往上一层，问题变了。不再是「我说什么」，而是「此刻这个窗口里，该装哪些东西，模型才解得动这道题」。

这就是 context engineering。检索什么资料、把长文摘成什么样、哪些过时信息要清出去。它管的不是你的话，是模型当下能看到的整个视野。

这层之所以单独成一层，是因为模型的窗口有限，塞错东西比塞少东西更糟。给它一窗户的噪音，再好的提示词也白搭。

第三层：Harness，单次运行的全套武装

再往上，到 harness engineering。这层管的是 agent 跑一次，身上得带什么装备。

给它哪些工具、允许它碰哪些动作、何时加载上下文、失败了怎么收场、什么状态才算「完成」。harness 只管 agent 在一次运行里能做什么、被拦在哪。

这层我在另一本书里整本展开过，这里不重复。你只要记住它的位置：harness 是 loop 的下一层。它把 agent 的单次运行武装好，但它不负责让这次运行自动重来。

第四层：Loop，让它自己一遍遍跑

最上面，loop engineering。前三层做完，你手里有了一个装备齐全、能漂亮跑完一次的 agent。但它跑完就停了，等你再敲下一拍。

Loop 这层，就是把「等你敲」这件事也自动化掉。Addy 那句话说得最干净。

原文：Loop engineering sits one floor above the harness.

循环工程，坐在 harness 的上一层楼。下面那层武装单次运行，这层负责让它一遍遍自己跑。

怎么个跑法，Addy 也给了画面：它在定时器上跑、孵化小帮手、自我喂食。原话三个动词，我一个个解释。

上一层楼，到底多了什么

harness 和 loop 经常被混在一起说，但它俩管的事差着一层。我用 Addy 那三个动词把界线划出来。

「在定时器上跑」。harness 武装的是一次运行，得你来触发。loop 给它装了个定时器，到点自己醒、自己开工，不用你按按钮。这是「自己一遍遍跑」的第一层意思。

「孵化小帮手」。一个 loop 转起来，会自己分出子 agent 去干并行的活。比如一个起草修改，另一个专门挑刺审查。harness 关心单个 agent 怎么武装，loop 关心一群 agent 怎么协作、谁监督谁。

「自我喂食」。这是最关键的一点。loop 跑出来的结果，会被它自己当成下一轮的输入。昨天发现的问题写进一个文件，今天醒来先读这个文件，接着干。它有了跨越单次对话的记忆，所以才叫循环，而不是重复运行了很多次的一次性任务。

Addy 有句话点破了这层关系：自动化才让一个 loop 成为真正的 loop，而不是你手动跑过一次的某次运行。少了「自己醒、自己接力」，剩下的就只是把 harness 手动按了很多遍。

为什么非得分这么清

你可能会问，分这么细有必要吗，不就是越来越自动嘛。

有必要。因为每一层的失败方式不一样，能说不的检查也得装在不同的地方。

prompt 错了，你当场就看见，改一句就行。context 装错了，模型答偏，你也容易察觉。但到了 loop 这层，它在你睡觉时自己跑、自己改你没看过的代码、自己把错误也一并喂给下一轮——出问题你可能几天后才发现。

层次越高，你离现场越远，犯的错就攒得越久。这也是为什么 loop engineering 真正的难点，从来不是把循环搭起来，而是往循环里放一个能拦住它的东西。这个话题，是后面几章的重头戏。

记住这张栈就够了：prompt 管一句话，context 管一窗户，harness 管一次跑，loop 管让它自己跑下去。你的位置，正随着这张栈一层层往上挪，从写话的人，挪到造循环的人。

§03 一个循环的五个动作

The Five Moves of One Loop

把一个 loop 转一圈拆开看，里面其实只有五个动作。看懂这五个，你就看懂了所有 loop。

循环这个词容易让人想偏。一听「循环」，脑子里浮现的是一段反复执行的代码，转个不停。

但 loop engineering 里的循环不是那种空转。它每转一圈，都在干一件具体的活：发现一件值得做的事，把它交给 agent，验证结果对不对，把状态存下来，然后决定下一步。

五个动作：发现、交付、验证、持久化、调度。少一个，loop 就转不起来，或者转起来也是空转。

用一个真实例子串起来

Addy Osmani 在 Google Chrome 团队，他给自己搭了一个早上的 triage loop。这个例子被反复引用，因为它够具体，每个动作都能落到实处。

整个流程是这样的。早上一个 automation 自动跑起来，不用他点。一个 triage skill 去读昨天 CI 跑挂的测试、还开着的 issue、最近的 commit。它把分诊结果写进一个 markdown 文件或者 Linear 看板。每个值得动手的发现，开一个隔离的 worktree。一个子 agent 起草修复，第二个子 agent 对照项目的 skill 和测试做审查。connector 自动开 PR、更新 ticket。处理不了的进收件箱等人。最后那个状态文件留着，第二天接着这里跑。

这个 loop 的妙处，是它每个环节都没让人插手，但又在该停的地方停下来等人。能自动的全自动，处理不了的老老实实进收件箱。下面把这一圈，按五个动作拆开看。

动作一：发现（discovery）

loop 转一圈，第一件事是搞清楚「这一圈该干什么」。

在 Addy 的例子中，发现这一步是 triage skill 干的。它去读三样东西：CI 失败、open issues、最近 commits。这三样合起来，就是「昨天到今天，系统里出了哪些值得管的事」。

发现的关键，是让 agent 自己去找活，而不是你把活喂给它。你要是每天早上手动列一遍「今天修这几个 bug」，那 loop 还是靠你启动的，没省下你。

这里有个容易被忽略的细节。Addy 强调，automation 里调用的是 skill，不是一大段指令。他的原话是，你触发的是 `skill-name`，而不是往一个没人会去更新的定时任务里贴一大段指令。skill 是固化下来的知识，发现逻辑变了，改 skill 就行。

发现这一步做得好不好，决定了整个 loop 的质量上限。它找出来的活要是没价值，后面四个动作做得再漂亮，也是在认真地做无用功。所以这一步不只是「读点东西」，而是要让 agent 学会判断什么值得做、什么可以放着，这恰恰是最难固化进 skill 的部分。

动作二：交付（handoff）

发现了活，得把它交出去让人干。这一步叫 handoff，意思是把任务从「调度系统」交到「干活的 agent」手里。

交付不是简单地喊一声「去修这个」。Addy 的做法是，每个值得做的发现，单独开一个隔离的 worktree。worktree 是 git 的一个机制，让多个 agent 在各自的工作目录里改代码，互不踩脚。

为什么要隔离，下一章会细讲。这里先记住一句话：**两个 agent 同时改同一个文件，跟两个工程师往同几行代码上提交，是一模一样的麻烦。**交付的时候就把它隔开，后面就少一堆冲突要收拾。

交付之后，真正干活的是子 agent。一个起草修复，写出第一版改动。这一版好不好，它自己说了不算。

这里有个设计取舍值得留意。**交付不是把活一股脑全推给一个 agent，而是先切成一个个隔离的小任务，再分头交出去。**任务切得越干净，后面验证和合并就越省事。一个发现对应一个 worktree，对应一份明确的改动，边界清楚，谁出问题都好定位。

动作三：验证（verification）

这是整个 loop 里最容易被偷工减料、也最不能省的一步。

Addy 的设计是，第一个子 agent 起草修复之后，第二个子 agent 来审查。第二个 agent 对照的是项目的 skill 和测试，instructions 跟第一个不一样，有时候连模型都换一个。

为什么非得换一个 agent 来审？因为**写代码的那个 agent，给自己的作业打分时太手软了。**让它评判自己刚写的东西，它倾向于自信地夸一遍——哪怕在人眼里质量明显一般。换一个 instructions 不同的 agent，专门挑刺，反而抓得住第一个 agent 自我说服放过去的问题。

Addy 说得很直接：一个 loop 难的不是 loop 本身，难的是往里面塞一个能说「不」的东西。一个没有真正检查的 loop，就是 agent 在那儿反复跟自己点头。

验证这一步，就是那个能说「不」的东西。它决定了 loop 是在真干活，还是在自我催眠。

动作四：持久化（persistence）

验证通过，结果得落到一个能活过这次对话的地方。这就是持久化。

Addy 用的是两样东西。改动本身，通过 connector 自动开 PR、更新 ticket，落进代码仓库和工单系统。处理不了的发现，进 triage 收件箱，等人工。还有一个贯穿始终的状态文件，记着「转到哪了」。

持久化的意义在于，loop 的记忆不能只活在当前的上下文窗口里。上下文一清空，agent 就忘干净了。但写进 markdown 或者看板的东西不会忘。这一点下一章讲组件时会再展开，那里有句话说得好：agent 会忘，仓库不会。

没有持久化，loop 每转一圈都像第一次睁眼，永远从零开始。有了它，今天没干完的，明天能接着干。

动作五：决定下一步 / 调度

前四个动作，发现、交付、验证、持久化，合起来是「转一圈」。但转一圈不等于一个 loop。

真正让它成为「循环」的，是第五个动作：调度。

Addy 的 triage loop 是早上自动跑的，不用他每天去点。那个状态文件，让今天没处理完的发现留到明天，明天的那一圈自动接着这里开始。是调度，把「我手动跑了一次」变成了「它自己一圈一圈在转」。

Addy 有句话点破了这件事：automation 才是让一个 loop 成为真正的 loop，而不只是你跑过一次的某一次运行。

原话：Automations are what make a loop an actual loop and not just one run you did once.
自动化，才是让循环成为真正循环的东西，而不只是你跑过一次的那一次。

没有调度，前面四个动作做得再漂亮，也是一次性的手工活。你今天兴致来了跑一遍，明天忘了就断了。加上调度，它才在定时器上不停地转，替你把这件事一直做下去。

五个动作，一句话各自记住

把这五个动作放在一起，loop 的骨架就清楚了。

动作	干什么	在 triage loop 里
发现	自己找出这圈该做的事	skill 读 CI 失败 / issue / commit
交付	把任务隔离着交给 agent	每个发现开一个 worktree
验证	换个 agent 说「不」	第二个子 agent 对照测试审查
持久化	把状态写到对话之外	开 PR + 收件箱 + 状态文件
调度	让它一圈圈自动转	早上 automation 自动跑

这一章拆的是动作，是「转一圈发生了什么」。下一章拆的是零件，是「搭一个能转的 loop，你手里得有哪几样东西」。两件事是一体两面：动作描述运转，零件描述材料。

§04 六个零件：搭一个 Loop 需要什么

Six Parts: What It Takes to Build a Loop

上一章拆的是动作，这一章拆的是零件。Addy 把搭一个 loop 需要的东西，归成六样。少哪一样，loop 都会在某个地方瘸。

动作描述的是「转一圈发生了什么」，零件描述的是「你手里得攥着哪些东西，才转得起来」。

这两件事对得上。发现靠 skill，交付靠 worktree，验证靠子 agent，持久化靠 memory，调度靠 automation。这一章就把六个零件一个个摊开，每个讲清三件事：它是什么、为什么需要、Addy 怎么说的。

零件一：Automations（自动化 / 调度）

Automation 是那个让 loop 自己动起来的東西。它挂在一个时间表或者触发器上，到点了就自动发现、自动分诊，不用人在旁边盯着按。

为什么它排第一？因为它是「循环」这个词的根。没有调度，你手里那套东西只是一次运行，不是一个 loop。上一章 Addy 的 triage 早上自动跑（如 § 03 所述），靠的就是 automation。

原话：Automations are what make a loop an actual loop and not just one run you did once.
自动化，才是让循环成为真正循环的东西，而不只是你跑过一次的那一次。

注意一个细节。automation 里该触发的是一个 skill，不是一整面墙的指令。你贴一大段 prompt 进定时任务，没人会再去更新它；触发一个有名字的 skill，逻辑变了改 skill 就行。

调度的形态不止一种。有的跑在你自己机器上，机器得开着；有的跑在云端，机器关了也照转。这两类差别挺大，后面专门有一章讲调度怎么选，这里先记住：automation 是 loop 的发动机，发动机不点火，剩下五个零件都只是摆设。

零件二：Worktrees（git worktree 隔离）

Worktree 是 git 自带的机制，让你在同一个仓库里开出多个互相独立的工作目录。每个 agent 在自己的 worktree 里改代码，互不干扰。

为什么需要它？因为 loop 一旦同时跑多个 agent，文件写冲突就是迟早的事。两个 agent 抢着改同一个文件，那种麻烦你大概率领教过。

原话：Two agents writing the same file is the exact same headache as two engineers committing to the same lines.

两个 agent 同时写一个文件，跟两个工程师往同几行代码上提交，是一模一样的头疼。

worktree 把并行 agent 隔开，让冲突在发生之前就被避开，而不是事后去合并一堆打架的改动。上一章交付那一步，每个发现单独开一个 worktree，就是这个零件在干活。

这个零件的价值随并行规模放大。一个 loop 同时只跑一个 agent，worktree 可有可无。可一旦你想让它早上同时修五个 bug，五个 agent 各改各的，没有隔离就是五份改动搅在一起，最后谁也不知道哪行是谁写的。worktree 让并行从「能跑但乱」变成「能跑且干净」。

零件三：Skills（用 SKILL.md 复用知识）

Skill 是把项目知识固化成一个文件（SKILL.md），让 agent 每次都能直接取用，不必每一圈都重新推一遍上下文。

为什么需要？因为不固化，你就得在每次运行里重新告诉 agent「这个项目是怎么回事、规矩是什么、坑在哪」。这种反复交代的成本，Addy 给了个名字，叫意图债（intent debt）。

skill 就是在还这笔债。把一次想清楚的东西写下来，之后每圈都省掉重新推导。发现这一步为什么是 triage skill 而不是一段裸 prompt，原因也在这——skill 能复用、能维护，prompt 墙不能。

零件四：Plugins & Connectors（MCP 连接器）

Connector 是让 loop 接到外部世界的接口。issue tracker、数据库、staging 环境的 API、Slack，都靠它连进来。技术底座是 MCP（Model Context Protocol）。

为什么需要？因为一个只能看见文件系统的 loop，能干的事太少了。它得能读工单、查数据库、开 PR、发通知，才算真的在替你跑活。

原话：A loop that can only see the filesystem is a tiny loop.

一个只能看见文件系统的循环，是个很小的循环。

MCP 还有个好处：跨工具兼容。你在 Codex 这边写的 connector，搬到 Claude Code 那边常常能直接用。持久化那一步里，connector 自动开 PR、更新 ticket，就是这个零件在收尾。

换个角度看，connector 决定了 loop 的「视野半径」。只接文件系统，它能看的就是几个目录。接上 issue tracker，它知道还有哪些活没干；接上数据库，它能查真实数据；接上 Slack，它能把结果告诉你。每多接一个外部系统，loop 能自主完成的链条就长一截。

零件五：Sub-agents（生成者和评判者分开）

子 agent 的关键用法，是把「写东西的」和「评判东西的」拆成两个。一个 agent 负责生成，另一个 instructions 不同、有时模型也不同的 agent 负责挑刺。

为什么非得分开？因为同一个 agent 既当运动员又当裁判，裁判会偏心。

原话： The model that wrote the code is way too nice grading its own homework. A second agent with different instructions and sometimes a different model catches the stuff the first one talked itself into. 写代码的那个模型，给自己的作业打分时太客气了。换一个指令不同、有时模型也不同的 agent，能抓住第一个自我说服放过去的东西。

验证之所以靠谱，就在于评判者跟生成者不是同一个。上一章那个「能说不的东西」，落到零件上，就是这第二个子 agent。

有个反直觉的点值得记一下。你可能觉得，要让 agent 学会批判自己写的东西，得花大力气调教生成者。但实践下来正相反：把一个独立的评判者调得足够挑剔，比让生成者对自己的作品狠下心来，容易得多。这也是为什么 loop 里宁可多养一个 agent，也不让一个 agent 自审自查。

零件六：Memory（持久化状态）

第六个零件，Addy 把它叫 Memory，不叫 Persistent State。它是 loop 活在单次对话之外的记忆，一份 markdown 文件，或者一块 Linear 看板。

为什么需要？因为 agent 的上下文窗口一清空，它就什么都不记得了。loop 要想今天接着昨天跑，记忆就不能只待在上下文里，得落到磁盘上。

原话： The agent forgets, the repo doesn't. The memory has to be on disk and not in the context. agent 会忘，仓库不会。记忆得待在磁盘上，不能待在上下文里。

持久化那一步写进 markdown 或看板的状态文件，就是这个零件。它让 loop 有了连续性。没有它，每一圈都是失忆后重新睁眼。

这里有个细微但重要的区分。memory 不是上下文。上下文是 agent 这一轮能看到的東西，会随着窗口刷新被冲掉。memory 是写在磁盘上的东西，跨轮、跨天、跨整个对话生命周期都还在。一个 loop 想长期跑，靠的不是把上下文撑大，而是把该记的东西沉到磁盘上，需要时再读回来。

六零件速查表

六个零件，对应五个动作，一张表收齐。

零件	是什么	对应动作	一句话原话
Automations	挂在时间表/触发器上自动跑	调度	make a loop an actual loop
Worktrees	隔离并行 agent 的工作目录	交付	same headache as two engineers
Skills	固化项目知识、还意图债	发现	fire \$skill-name, not a wall of instructions
Connectors	MCP 接外部系统	持久化 / 发现	only see the filesystem is a tiny loop
Sub-agents	生成者与评判者分离	验证	too nice grading its own homework
Memory	磁盘上的持久状态	持久化	the agent forgets, the repo doesn't

六个零件齐了，一个能自己转的 loop 就有了骨架。Automation 让它动，worktree 让它不打架，skill 让它不重复劳动，connector 让它看得见外面，子 agent 让它能自我纠错，memory 让它记得住。

但搭起来只是开始。一个能转的 loop，转好还是转坏，是另一回事。同一套零件，两个人搭出来，结果可能完全相反。这就是后面要谈的代价和反模式。

§05 生成器与评判器：为什么写代码的 AI 不能给自己打分

Generator and Evaluator

一个 loop 最难的地方，不是让 agent 跑起来，而是放一个能说「不行」的东西进去。可写代码的那个 agent，恰恰是最不会说「不行」的。

它总夸自己

你让一个 agent 写完一段代码，再让它评价自己写得怎么样。它会怎么回答？

Anthropic 的工程师 Prithvi Rajasekaran 在一篇官方博客里给了答案。他在做长时间运行的应用时观察到一个稳定的现象。

原文：when asked to evaluate work they've produced, agents tend to respond by confidently praising the work—even when, to a human observer, the quality is obviously mediocre.

让 agent 评价自己产出的东西，它往往会自信地夸一通，哪怕在人看来质量明显很一般。

这不是模型不够聪明，是它在给自己的作业打分。写代码的那个上下文里，已经塞满了「我为什么这么写」的理由。它看自己的代码，看到的不是结果，是一路推导过来的自我说服。

你我都干过这事。自己写的文章，回头读一遍，越读越顺。问题不是文章好，是你脑子里已经有了它该是什么样。读的时候，你看的不是纸上的字，是脑子里那个版本。

放到 loop 里，这个毛病会被放大。一个 loop 之所以值钱，是因为它能无人值守地跑很多轮。可如果每轮的「这一版行不行」都由刚写完的 agent 自己拍板，那它每一轮都在给自己点头。跑得越久，攒下的自我表扬越多，离真实质量越远。一个没有真正检查的 loop，本质上是 agent 在重复地跟自己达成共识。

改一个怀疑论者，比改一个谦虚的作者容易

Rajasekaran 试过让生成代码的 agent 对自己更挑剔。不太行。

原文：tuning a standalone evaluator to be skeptical turns out to be far more tractable than making a generator critical of its own work.

调一个独立的评判器，让它变得怀疑，比让生成器自我批判要容易得多。

区别在结构，不在措辞。你没法靠一句「请严格一点」让作者跳出自己的视角。但你可以换一个 agent，给它完全不同的指令，让它从零开始看这段代码。它没参与写，就没有那套自我说服。

这是个反直觉的结论。我们的本能是去改 prompt：写得再凶一点，逼生成器自己挑刺。可 Rajasekaran 的经验是，这条路越走越费劲，性价比远不如另起一个 agent。**自我批判难，难在批判的对象和批判者共用一个上下文；换 agent 容易，是因为你用结构强行造出了一个外人。**

这套思路 Rajasekaran 说得很直白，他借鉴的是 GAN。

原文：I designed a multi-agent structure with a generator and evaluator agent.

生成对抗网络（GAN）里，一个网络负责造，一个网络负责挑刺，两个对着干，造的那个越来越像真的。搬到 agent 上，就是一个 generator 写，一个 evaluator 审，结构上把「写」和「判断写得好不好」彻底分开。

这正是上一章 Addy 框架里子 agent 那一格讲的事。Addy 的说法更不留情面（原话见 § 04 零件五）：写代码的模型，给自己批作业态度太好。换第二个 agent，换一套指令，有时候连模型都换掉，它能抓出第一个 agent 自己绕进去的那些坑。

评判器要会动手，不只是会读

光换个 agent 还不够。如果 evaluator 只是把代码读一遍，它判断的还是「代码看起来对不对」，不是「跑起来对不对」。

Rajasekaran 在前端任务里的做法值得抄。**evaluator 接上 Playwright MCP，自己打开页面、点按钮、截图、查 DOM，像一个真人 QA 那样去用这个东西。**

这一下就把判断的依据从「我觉得这段 JSX 没问题」变成了「我点了登录按钮，页面跳转了，截图在这」。一个会动手的评判器，看的是行为，不是意图。

有意思的是，Addy 提到换 agent 的时候还顺手换了模型（sometimes a different model）。同一个模型，哪怕换了指令，思维的盲区往往还在原地。换一个底座，连它习惯性绕过去的坑都可能换一批。评判这件事，差异越大越值钱。

实践者在写 evaluator 指令时，常常把它往最坏处校准。一种常见的措辞是让评判器默认「这段代码是坏的，除非被证明能跑」（assume the code is broken until proven otherwise）。这句不是 Anthropic 的官方说法，是社区里摸出来的经验。但方向对——**评判器的默认态度应该是怀疑，不是信任。**生成器已经够信任自己了，评判器再客气，这个 loop 就等于 agent 自己跟自己点头。

落到产品：/goal 把评判器用在了停止条件上

讲到这你可能觉得，generator 和 evaluator 听着像架构图上的两个方框，离手有点远。其实 Claude Code 里已经有一个命令把这个结构做成了产品原语，叫 `/goal`。

`/goal` 的用法是给 agent 一个条件，让它一直跑到条件满足为止。比如：

```
/goal all tests in test/auth pass and the lint step is clean
```

关键在「谁来判定满足了没有」。不是正在干活的那个 agent 自己说「我觉得可以了」。

官方文档： After each turn, a small fast model checks whether the condition holds. If not, Claude starts another turn instead of returning control to you. completion is decided by a fresh model rather than the one doing the work.

每跑完一轮，一个又小又快的模型来检查条件成立没有。不成立，Claude 就再跑一轮，而不是把控制权交还给你。**是否完成，由一个全新的模型判定，不是由干活的那个判定。**

这就是 maker-checker（生产者-检查者）原则应用在了停止条件上。干活的 agent 是 maker，那个 fresh model 是 checker。它没参与这一轮的工作，没有自我说服的包袱，问它「测试真过了吗」，它只能去看测试结果。底层实现是一个基于 prompt 的 Stop hook，每轮拦一下，判定不通过就不放行。

maker-checker 不是 AI 发明的。银行里转一笔大额，录入的人和复核的人必须是两个人，这条规矩存在几十年了，原因和这里一模一样：经手的人会信任自己的操作，复核才有意义。/goal 做的事，本质就是把这条老规矩塞进了 agent 的循环里。每一轮的「我做完了」，都得过一道不是它自己的检查。

谁在判定「完成」		问题 / 优势
agent 自评	干活的那个 agent	带自我说服，倾向于夸自己
/goal	每轮换一个 fresh 小模型	无包袱，只能看客观条件

要分清一件事。/goal 是 Claude Code 的命令名。Codex 上没有这个命令，它走的是 Automations 加 agents 的配置去实现同类能力，结构一样，叫法不一样。并列着写的时候别把命令名安错门。

还有个更容易混的：别把 /loop 当成这个。/loop 是按时间间隔定时重跑，是调度，跟「跑到条件满足为止」是两码事，下一章专门讲它。

一个 loop 的下限，是它的评判器

把这一章收一下。一个能跑的 loop 不难搭，难的是给它配一个靠谱的评判器。

generator 的水平决定 loop 能产出什么，**evaluator 的水平决定 loop 不会产出什么**。没有后者，loop 跑得越欢，攒下的烂代码越多，因为没人喊停。

结构上分开生成和评判，把评判器调成怀疑论者，让它会动手验证而不只是读，再把判定权交给一个没参与干活的 fresh 模型。这四步，就是让一个 loop 长出「说不」的能力的全部。

下一章我们看三个真在跑的 loop，看这套东西在别人睡觉的时候是怎么干活的。

§06 让循环在你睡觉时跑：三个真实的 Loop

Three Real Loops

前面讲的都是「应该怎么搭」。这一章只看「谁真的搭好了在跑」。三个公开案例，从一个人的早晨，到一家公司每周一千多个 PR。

这三个案例规模差得很远，但骨架是一样的：有一个触发器替你按下开始，有一套约束保证它别跑飞，最后留一个人类的检查口。看完你会发现，所谓「在你睡觉时跑」，拼的从来不是模型多强，而是这副骨架搭得多稳。

一、Addy 的早晨：一个真在跑的 triage loop

Addy Osmani 描述过他自己的一个 loop。不是设想，是每天早上自动发生的事。

这套流程 §03 已经拆过：天亮一个 automation 自己醒来，读 CI 和 issue，开 worktree，子 agent 起草和审查，过了就自动开 PR，没把握的丢进收件箱等人，状态写进文件留给第二天。这里只点一个 §03 没强调的细节：**automation 调用的是一个 skill，不是把一大段指令贴死在日程里**。Addy 的原话挺扎心。

原文：you fire \$skill-name instead of pasting a giant wall of instructions into a schedule that nobody will ever update.

你触发一个 \$skill-name，而不是把一大墙指令贴进一个排程里。那种东西贴进去，以后没人会再去更新它。

这是个人能跑起来的 loop 的样子。一个人，一台机器，每天早上替你把脏活先过一遍。

二、Stripe 的 Minions：每周 1300 个 PR

把 loop 推到企业规模，最值得看的是 Stripe 的 Minions。

数字先放这：**每周合并 1300 多个 PR，没有一行是手写的**。这些细节来自 Stripe 工程师 Steve Kaliski 在播客《How I AI》里的讲述。

触发方式很轻。你在 Slack 里 @ 一下 Minion bot，或者给某条消息加个 emoji 反应，发完就不用管了（fire-and-forget）。

但真正让它靠谱的，不是触发轻，是 LLM 醒来之前那一段。**在模型开始思考前，一个确定性的 orchestrator 先把上下文备齐**：扫消息里的链接、拉 Jira、找文档、用 Sourcegraph 加 MCP 把相关代码搜出来。等 agent 上场，桌上已经摆好了它要的所有材料。

这一步为什么重要？因为让 LLM 自己去找上下文，是最不可控的环节。它可能搜错、可能漏、可能为了凑答案现编。Stripe 的做法是把「找资料」这种能写死规则的活，从 LLM 手里拿走，交给确定性代码去办。LLM 只负

责它真正擅长的那部分：在材料齐全的前提下写代码。**能用确定性逻辑解决的，绝不交给概率模型。**这条线划在哪，直接决定这个 loop 靠不靠谱。

最反直觉的一点在底层。Minions 不是建在某个更强的模型上，它是开源工具 **Goose** 的一个 **fork**。核心论点就一句：AI 的可靠性来自约束的质量，不是模型的大小。

它的架构是六层，确定性的 gate 和 LLM 的创造步骤交替咬合。agent 写完代码，一段硬编码的 pipeline 跑 linter，agent 跳不过去；agent 去修 lint；修完，硬编码的步骤执行 git commit。沙箱用的是 Devbox，跑在 EC2 上，「cattle not pets」，用完即弃，不当宠物养。

「cattle not pets」这句运维老话，放在这里特别贴切。每个 agent 的工作环境都是一头随时可以换掉的牛，不是一只养着的猫。坏了不修，直接起一个新的。这种设计让一千多个 agent 同时跑而不互相踩到脚，因为没有哪个环境是「珍贵」的、动不得的。约束的质量，体现在这些不起眼的工程决定里，而不在模型参数上。

环节	谁来做
触发	人（Slack @ 或 emoji）
备上下文	确定性 orchestrator（非 LLM）
写代码	LLM agent
跑 linter / commit	硬编码 pipeline（agent 跳不过）
review 这 1300 个 PR	人

注意最后一行。**人没退场，人换了工位。**这 1300 个 PR 仍然由工程师 review，时间从「写」挪到了「审」。这恰好接上一章的话：loop 越能产出你没写的代码，能说「不」的那一环就越关键。Stripe 把一半「不」交给硬编码 gate，另一半留给人。

三、那「睡觉时跑」到底靠什么

「让循环在你睡觉时跑」是这一波最性感的一句话。但得说准。

本地的 /loop 也好，桌面的定时任务也好，**它们都要你的机器开着。**机器一关，loop 就停了。真要做到关机也跑、睡着也跑，正解是别的东西：Cloud Routines，或者 GitHub Actions 的 schedule 触发。

	Cloud Routines	Desktop 定时任务	/loop
跑在哪	Anthropic 云	你的机器	你的机器
需要开机吗	否	是	是
需要开着会话吗	否	否	是
最小间隔	1 小时	1 分钟	1 分钟
能看本地文件吗	否 (fresh clone)	能	能

读这张表的方式很简单。要频繁、要看本地文件，用本地 /loop，代价是机器得开着。要关机也跑、不依赖本地状态，上云，代价是最小间隔一小时、每次都是干净的 clone。没有一种调度通吃，选哪种取决于你这个 loop 是粘本地还是能离开本地。

举两个实际场景。你想让一个 loop 每分钟盯一下本地正在跑的开发服务器，那只能是本地 /loop，云端看不到你机器里那个进程，间隔也下不到一分钟。反过来，你想让它每天凌晨三点扫一遍仓库的 open issue、该提 PR 提 PR，这种活根本不该绑在你的笔记本上。笔记本会合盖、会断电、会被你带出门。这时候 GitHub Actions 的 schedule 触发或者 Cloud Routines 才是正解，它们跑在一台永远醒着的机器上，你睡你的。

「睡觉时跑」之所以容易被讲歪，就是因为很多人把本地 /loop 当成了它的全部。本地 /loop 是「我在场时让它替我多跑几轮」，云端调度才是「我不在场它也照跑」。两件事，别混。

排程本身没什么神秘。/loop 用的是标准的 cron 表达式，五个字段，最小粒度一分钟；嫌它烦，设个 CLAUDE_CODE_DISABLE_CRON=1 就全关了。这层东西早就是运维的常识，loop engineering 没重新发明它，只是把 agent 挂了上去。

Codex 这边对应的是 Automations 标签页。可以设 daily、weekly，也能用 cron 自定义。它跑在一个专用的 background worktree 里，结果汇进一个 Triage 收件箱等你回来看。一样的道理，本地的 automation 也要开机，想关机跑得靠云，Codex 自己的云端 Jobs 还在规划里。

这三个案例放一起，其实是一条线：个人能搭（Addy 的早晨），企业能规模化（Stripe 的一千个 PR），调度有现成的层可选（本地还是云）。loop engineering 不是一个未来概念，它已经在很多人的机器上、很多家公司的 CI 里，每天跑着。

有几个流传很广的数字，你看到时最好谨慎对待。比如「Anthropic 约九成 Claude Code 由它自己写」、「Nubank 迁移百万行 ETL 提效 12 倍」这类，多是二手综述，姑且当个参照，别太当真。这一章三个案例的细节都能追到一手出处，这比一个唬人的大数字更经得起较真。

下一章我们换个角度，看这套东西的代价。一个无人看管的 loop，也是一个无人看管地在犯错的 loop。

§07 代价：验证债、理解腐烂、token 失控

The Costs

循环替你干活，听着全是好处。但它也在悄悄替你欠债。这章讲四笔账，每一笔都不会自己消失。

循环自己跑，也在自己犯错

前面几章我一直在讲循环能替你做什么事。这章我得把另一半摆出来。一个能自己跑的循环，同时是一个能自己犯错的循环。

Addy 这句话，我读到的时候停了一下。

原文：A loop running unattended is also a loop making mistakes unattended.

一个没人看着的循环，也是一个没人看着犯错的循环。它跑得越欢，错也错得越安静。

第一笔账叫验证债。你让循环自动开 PR、自动改代码、自动合并，每一步都省了你的时间。但省下来的时间不是白来的，它变成了一堆「还没人验证过」的产出，堆在那儿等你还。

这笔债的麻烦在于，它不会立刻找你。循环今晚改了二十个文件，测试全绿，PR 全开，看起来漂亮极了。问题藏在那些测试没覆盖到的地方，藏在「能跑」和「对」之间的缝里。它们安静地躺着，攒到某个上线的早上，一起爆给你看。你以为省下的是时间，其实只是把还债的日子往后推了。

为什么会欠下这笔债？因为加一个 agent 很容易，加一个能拦住 agent 的东西很难。Addy 把这件事说得很重。

原文：The hard part of a loop is not the loop. It is putting something inside it that can say no.

循环的难点不在循环本身。难的是往里面放一个能说「不」的东西。一个没有真正检查的循环，不过是 agent 在那儿反复跟自己点头。

怎么防？给循环装一个独立的评判者，而且这个评判者得跟干活的那个不是同一个。第二个 agent、不同的指令、最好换个模型，专门挑刺。让写代码的 agent 自己打分，等于让它自己夸自己。这一层在前面讲生成器和评判器时已经铺过，验证债就是它没装好时的账单。

你以为你懂这个 repo，其实你不懂了

第二笔账更隐蔽，叫理解腐烂。

循环每天替你写一堆你没写过的代码。代码能跑、测试能过、PR 能合。表面上一切正常。但有个东西在背后慢慢变质：你对这个项目的理解。

原文： The faster the loop ships code you did not write, the bigger the gap between what exists and what you actually get.

循环交付你没写的代码越快，「实际存在的东西」和「你真正理解的东西」之间的差距就越大。Addy 还补了一句，你的理解会不会腐烂，取决于你允不允许它腐烂。

为什么会发生？因为读代码比写代码无聊，而循环把写的部分全包了。你不再被迫一行行读过去，自然就不读了。代码库在长大，你脑子里的那张地图却停在三个月前。等出事的那天，你打开文件，发现自己像在看别人的项目。

这个差距最坑人的地方，是它平时完全不报警。项目跑得好好的，你也觉得自己挺懂这套东西。直到某个 bug 钻进一个你从没读过的角落，循环修不动了，得你亲自上。这时候你才发现，你对自己的项目已经是个外人。**理解腐烂不会发出声音，它只在你最需要理解的那一刻，让你发现理解已经没了。**

怎么防？强迫自己定期读循环的产出，别只看「测试过了」就合。挑几个改动，自己讲一遍它干了什么、为什么这么改。讲不出来，就是你的地图该更新了。

循环跑顺了，人就懒得有意见了

第三笔账，是前两笔的心理版本，叫认知投降。

原文： When the loop runs itself its very tempting to stop having an opinion and just take whatever it gives back.

当循环自己跑起来，你会很想停止有自己的判断，它给什么你就收什么。这句话戳中的，是一种特别舒服、又特别危险的状态。

验证债和理解腐烂还是技术问题，认知投降是态度问题。前两个是你没时间管，这个是你不想管了。**循环越可靠，你越容易把判断这件事整个外包出去。**

为什么会发生？因为持续有判断很累。每个 PR 都要自己想一遍对不对，比直接点「合并」累多了。循环跑得越稳，你就越倾向于相信它，越相信就越不看，越不看就越没法判断。这是个会自我加速的滑坡。

怎么防？保住一条底线。循环可以替你执行，不能替你拿主意。它递回来的东西，你至少得有能力说一句「这个不对」。判断不必每次都改，但能力得在。说不出口的那一天，不是循环变聪明了，是你悄悄退场了，只是还没人通知你。

账单可能比你想象的厚很多

最后一笔是真金白银的账，叫 token 失控。

前三笔欠的是隐性的债，这一笔是直接打在账单上的钱。一个自主跑的循环，消耗有多少，很难提前算准。

原文： You absolutely have to be careful about token costs.

你必须非常小心 token 成本。Addy 还点了一句关键的：用量会剧烈波动，取决于你是 token 富人还是 token 穷人。同一个循环，富人跑得起，穷人可能一夜醒来发现账单爆了。

为什么不可预测？因为循环会自己孵化小帮手、自己重试、自己在定时器上一遍遍跑。你写的是逻辑，跑出来的是次数，而次数乘以单价才是钱。一个 bug 让它空转一整晚，你睡醒看到的就不是修好的代码，是一张陌生的账单。

怎么防？上线前给循环设几个硬上限：单次预算、每日预算、最大重试次数，到顶就停。这些数字不是为了省钱，是为了让一个空转的 bug 没法把整夜的额度烧光。**别等账单教你这一课，先用一个数字把天花板钉死。**

四笔账放在一起

这四笔账有个共同点：它们都不会在循环跑的当下报警。

代价	症状	一句话防它
验证债	产出堆着没人验，错误安静积累	装一个跟干活的不是同一个的评判者
理解腐烂	代码在长，你脑里的地图停了	定期读产出，讲不出就是该更新
认知投降	循环给啥收啥，懒得有意见	执行可外包，拿主意不行
token 失控	用量剧烈波动，账单不可预测	上线前钉死预算和重试上限

循环工程最迷人的地方，是它让一个人能干一个团队的活。最危险的地方，也是同一处——团队会互相 review、互相提醒、互相吵架，一个人加一堆循环，很容易变成没人吵架的回音壁。

这四笔账不是劝你别用循环。**是提醒你，省下来的时间得有一部分，花在替自己守住这四道关上。**下一章我们聊聊，守关这件事到底靠什么。

§08 当工程师，不只是按下启动键

Stay the Engineer

同一个循环，两个人造，结果可以截然相反。区别不在循环里，在造它的人手上。

同样的循环，相反的结局

这本书前面教了很多怎么造循环。这一章我想往回退一步，聊聊造循环的人。

Addy 在文章里写了一句话，我觉得是整篇里最该被记住的。

原文：Two people can build the same loop and get opposite outcomes.

两个人造出一模一样的循环，得到的结果可以完全相反。

这句话听着有点反直觉。循环是个系统，系统是中立的，同样的代码跑出来不该是同样的东西吗？但 Addy 说的不是循环本身，是循环之外那个人。

一个人用循环，是为了在自己已经吃透的事情上跑得更快。代码他读得懂，方向他拿得准，循环只是替他把手动的活包了。循环帮他扩大的，是他本来就有的判断。

另一个人用同样的循环，是为了不必再去理解。看不懂没关系，循环会写；判断不了没关系，循环会合。他用循环把「理解」这件麻烦事整个绕过去了。

两个人，同样的工具，一个在加速理解，一个在逃避理解。半年后再看，一个人变得更强，另一个人变成了一台自己都不知道在跑什么的机器的看门人。

这件事我觉得值得多想一层。我们总以为工具的好坏由工具决定，强工具配出强结果。但循环不是这样的工具。它太强了，强到会把你带进它里面的那个东西原样放大。你带进去的是理解，它放大的理解；你带进去的是偷懒，它放大的偷懒。循环本身不偏向任何一边，它只是个忠实的乘号，乘的是你。

不稀缺的和稀缺的

循环让一件事变得极其便宜：生成。

代码、方案、PR、修复，都能批量造出来，几乎不要钱、不要等。当一样东西可以被无限生成，它就不再值钱。今天满地都是 AI 写的代码，它的稀缺性正在归零。

那什么还稀缺？

判断力。知道哪个方案是对的、哪行代码该拦下来、哪个产出虽然能跑但根上错了——这件事循环替不了你。

循环能生成一百个选项，但它没法替你选。或者更准确地说，它可以替你选，但它选的依据是「看起来合理」，不是「真的对」。这两者的差距，正好就是一个工程师存在的理由。

所以循环工程不是让判断力贬值，恰恰相反。它把所有不需要判断的活都拿走了，剩下的全是判断。你的价值被压缩、被提纯到只剩这一件事。这件事你做得好，循环放大你；做得差，循环放大你的差。

这其实是个挺残酷的放大器。过去你判断失误，顶多自己手写错一段代码，影响有限，还来得及发现。现在你判断失误，循环会忠实地、批量地、不知疲倦地把这个错误执行一百遍。它不会停下来问你「这真的对吗」，它只会跑得更快。**工具越强，对开你判断力的赌注就下得越大。**你不能再指望「过程慢」帮你兜底，因为过程已经没有慢这个挡了。

循环不懂你为什么造它

还有一层，关于谁来设计这个循环。

循环会忠实地执行你给的逻辑。但它不懂你当初为什么要造它，不懂你真正想要的是什么，不懂哪些地方你其实是想自己把关的。它只看见代码，看不见动机。

这意味着，循环的边界在哪、哪里该留一个人工的卡点、哪里该让它自己跑，这些决定没人能从循环里读出来。它们只活在造它的人的脑子里，得被事先想清楚、再一条条写进去。漏掉的那部分，循环不会替你补，它只会照着你写的跑，连同你忘了写的那些缺口一起跑。

你用什么心态去设计它，它就长成什么样。如果你是抱着「赶紧把自己解放出去」的心态造的，你会把所有检查都省掉，因为检查碍事。如果你是抱着「我还要在这儿当工程师」的心态造的，**你会主动给它留几道你能说不的关口，哪怕那意味着它慢一点。**

这两种心态造出来的循环，代码可能九成相同，差的就是那一两个卡点。可正是那一两个卡点，决定了半年后你是站在循环上面，还是被循环架空。前一种人留了门，他随时能走进去看看里面在发生什么。后一种人把门焊死了，图的是再也不用进去。等到非进去不可的那天，他发现自己已经没有钥匙了。

同一个循环的两种结局，差别就埋在这个起点里。

造循环，但要像一个打算留下来的人

Addy 的文章结在一句话上，我把它原样放在这儿，因为我想不出更好的收尾。

原文：Build the loop. But build it like someone who intends to stay the engineer, not just the person who presses go.

造那个循环。但要像一个打算继续当工程师的人去造它，而不是一个只负责按下启动键的人。

这两种人，外表看一样。都坐在循环外面，都让 agent 替自己干活，都享受着一个人干一个团队活的爽感。

区别在他们对自我的定位。一种人把自己定位成工程师，循环是他延伸出去的手，他始终知道手在做什么。另一种人把自己定位成按钮，按一下，等结果，结果好坏他也说不清。

循环工程给了每个人一个选择的机会。你可以用它把自己变强，也可以用它把自己变成一个多余的人。工具是同一个，按钮也是同一个。

我自己的体会是，这个选择不是造循环那天一次性做完的，它每天都在重做。你今天多读了一个 PR，多问了一句「这真的对吗」，你就还在工程师那一边。你今天图省事，闭着眼合了，你就往按钮那边滑了一格。**没有谁天生的工程师，也没有谁一劳永逸地是。它是个需要每天续费的身份。**

真正决定结局的，不是你造了多牛的循环，是你按下去的时候，心里还认不认自己是那个工程师。

§09 今天就动手：搭你的第一个 Loop

Build Your First Loop Today

前面讲了那么多原理，这一章只干一件事：带你从一行命令起步，一层层搭出一个真能自己跑的循环。

别想着一步到位

很多人一听 loop engineering，脑子里立刻浮现 Stripe 那种每周一千多个 PR 的流水线，然后觉得离自己太远，放弃了。

这是个误会。Stripe 那套是终点，不是起点。第一个 loop 应该小到几乎不像个系统，就是一个会定时帮你看一眼的小东西。

我的建议是，从一个 `/loop` 定时任务开始。它是 Claude Code 自带的命令，门槛最低，跑起来你立刻能感觉到「哦，原来是这么回事」。

第一步：跑一个 `/loop`

`/loop` 在 Claude Code v2.1.72 之后可用，作用就一个：按时间间隔，把同一件事重跑一遍。它有三种形态，你按需要挑。

三种形态：

`/loop 5m check the deploy`：固定 5 分钟一次

`/loop check the deploy`：Claude 自己决定节奏（1 分钟到 1 小时之间）

`/loop`：裸跑，执行内置的维护任务或你写在 `.claude/loop.md` 里的内容

时间单位是 s/m/h/d，cron 间隔最小 1 分钟。有两个细节你得记住，免得踩坑。

一是它是 session-scoped，**recurring 任务 7 天后过期**，不是有些教程里说的 3 天。二是它跑在你本机，机器关了就停。真要在你睡觉、关机时也跑，得换 Cloud Routines 或 GitHub Actions，这个后面会提。

第二步：让它读 CI 和 issue，先做 triage

光重跑一句话不算 loop，得让它读点东西、做点判断。最朴素的起点是一个 triage（分诊）任务。

你给它一个 prompt，让它每天早上去看三样东西：昨天的 CI 有没有失败、有哪些新开的 issue、最近的 commit 改了什麼。看完，挑出值得处理的，列个清单。

到这一步，你已经有了 loop 最核心的两个零件：一个定时触发的发现机制，加一个会替你过一遍信息的分诊脑子。Addy 把这种自动发现叫 automation。

Addy 说，是 automation 让循环成为真正的循环，而不只是你跑过一次的那次（原话见 § 03）。这句话点破了门槛：定时+自动发现，才算 loop 入门。

第三步：加一个状态文件，让它有记忆

分诊跑完，结果存哪？别让它留在对话窗口里，关掉就没了。开一个 markdown 文件，把每次的发现、处理到哪一步都写进去。

这个文件就是 loop 的记忆。Addy 的说法很形象：agent 会忘，仓库不会忘。记忆得落在磁盘上，不能只活在上下文里。有了它，今天没处理完的事，明天这个 loop 醒来还能接着干。

不想用裸 markdown 的话，换成 Linear 看板、issue 列表也行，道理一样：状态要活在单次对话之外。

第四步：加一个 evaluator，让它能说不

这一步是整个 loop 里最关键的，也是最容易被跳过的。

一个没人看着的循环，犯起错来也没人看着。你得在循环里塞一个能对它说「不」的东西，否则它就是在反复地自我点头。

Claude Code 里有个现成的工具叫 `/goal`（v2.1.139 之后可用），专门干这个。它跑到某个条件满足为止，而判断条件成立的，是另一个模型。

这个机制 § 05 拆过，判断完成的是另一个 fresh 模型，不是干活那个。

为什么非得换一个模型？因为干活的那个太容易夸自己。它写完代码，让它评自己写得怎么样，多半会自信地说不错。换个带着不同指令、甚至不同模型的评判者，才能抓出它说服自己糊弄过去的地方。用法很直接，比如：

```
/goal all tests in test/auth pass and the lint step is clean
```

第五步：加 worktree，让它并行

前面四步搭完，你已经有一个能自己跑的单线 loop 了。最后一步，是让它能同时干好几件事。

办法是 git worktree。Claude Code 用 `--worktree`（或 `-w`）给每个后台 agent 开一个独立 worktree，互不踩脚。

两个 agent 写同一个文件，跟两个工程师改同一行一样头疼（原话见 § 04）。隔离开，它们才能并行而不打架。到这儿，你的 loop 已经五脏俱全了。

工具现状速查（2026 年 6 月）

上面用的命令都是 Claude Code 的。Codex 也能做同样的事，只是入口和叫法不一样。别把命令名安错家。`/loop`、`/goal` 是 Claude Code 的，不是 Codex 的。

能力	Claude Code	Codex
定时调度	<code>/loop</code> （后台定时 worker）	Automations 标签页（daily/weekly + 自定义 cron）
跑到条件满足	<code>/goal</code>	—（靠 automation 重跑 + 判断）
并行隔离	<code>--worktree</code> / <code>-w</code>	专用 background worktree，结果进 Triage 收件箱
子 agent	Subagents / Agent Teams （ <code>.claude/agents/</code> ）	<code>.codex/agents/</code> TOML 定义
外部连接	MCP + Plugins（一键安装）	MCP connector
显式调技能	Skills（SKILL.md）	<code>\$skill-name</code>
关机也跑	Cloud Routines（跑在云端）	云端（规划中的 Codex Jobs）

MCP 这一行值得多说一句：它跨两边兼容，你在一边写好的 **connector**，挪到另一边常常能直接用。至于 Cursor 那套（Background Agents、subagents、Cloud Agents），能力路线类似，但版本细节我手上是二手信息，这里就不展开了，你要用的话自己核一下它最新的 changelog。

第一个 loop 检查清单

动手之前，对着这六条过一遍。缺哪条，你的 loop 就在那条上有风险。

要素	问自己
发现源	它定时去读什么？（CI / issue / commit / 收件箱）
状态文件	跨轮的记忆落在哪个磁盘文件上？
evaluator	有没有一个独立的、会说「不」的检查？
隔离	并行的 agent 是不是各自一个 worktree？
token 上限	设没设花费的天花板？跑飞了谁拦得住？
人工复核点	哪一步停下来等你看一眼，而不是一路自动到底？

这六条里，前两条决定你的 loop 能不能跑，后四条决定它跑起来会不会闯祸。新手最容易只搭前两条就上线，结果就是一个没人看着、也没人能拦的循环在那儿自我点头。

所以我的收尾建议很简单：**第一个 loop，宁可小，也要把那个会说「不」的检查和人工复核点装齐。**至于搭好之后怎么不被它反过来架空，§ 08 已经谈过，回去翻一翻就好。

这本书到这儿就讲完了。原理、零件、案例、代价，能说的我都说了。剩下的不在书里，在你的终端里。**现在就去搭你的第一个 loop。**

Loop Engineering

别再问我什么是循环工程



微信搜一搜

🔍 花叔

花叔 • AI Native Coder

我一行代码都不会写，却用 AI 做出了 AppStore 付费榜 Top 1 的小猫补光灯，写了多本技术书。

所有产品全部 AI 写的，我只负责想清楚要做什么。我始终相信：AI 生成内容不稀缺，稀缺的是判断力。

[B站：花叔v](#) • [公众号：花叔](#) • [X：花叔](#) • [YouTube](#) • [GitHub](#) • [官网](#)

Created by 花叔 • 2026年6月

所有橙皮书免费下载：huasheng.ai/orange-books